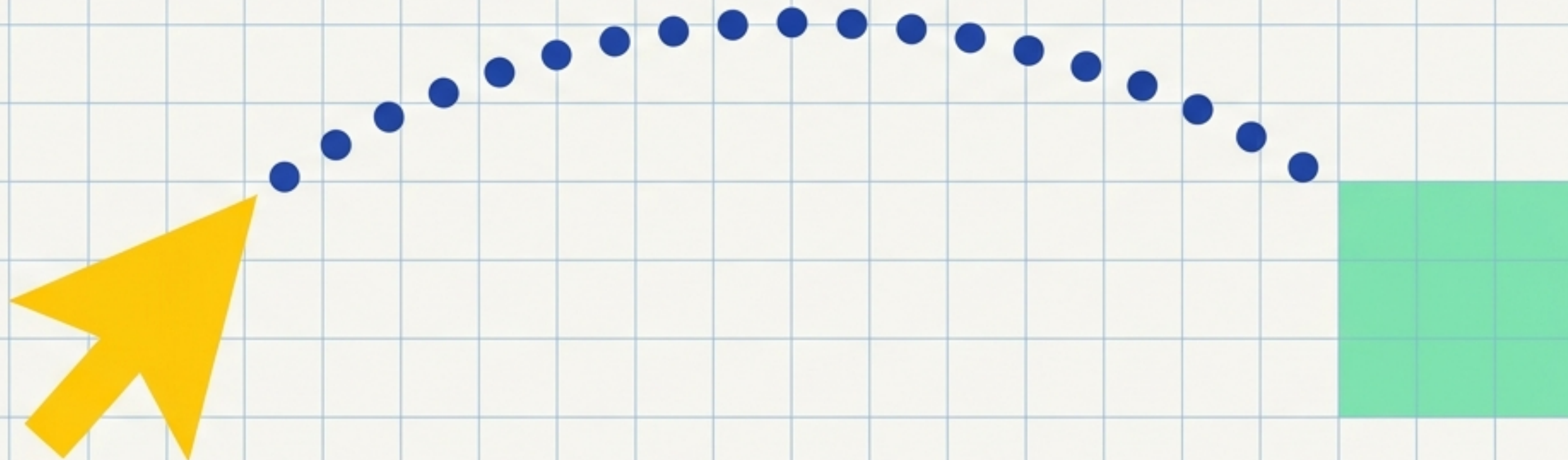


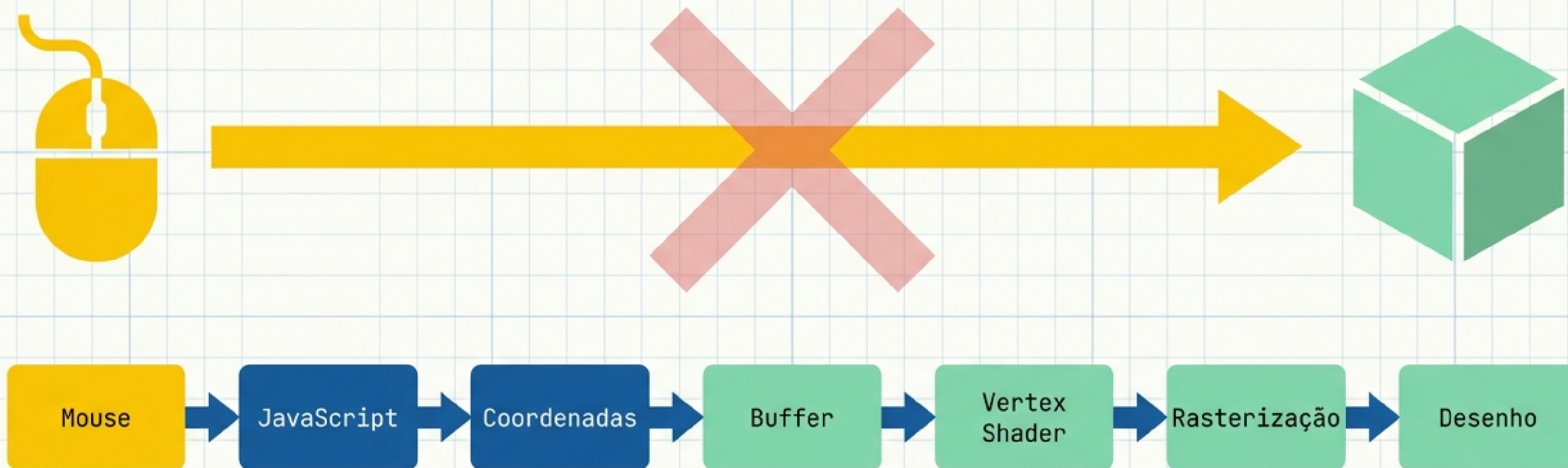
Do Clique ao Pixel



A arquitetura invisível entre o mouse e a GPU no WebGL

A Ilusão da Interação Direta

O mouse não interage diretamente com o WebGL. Ele é cego para a GPU. Quem faz a ponte é o JavaScript.

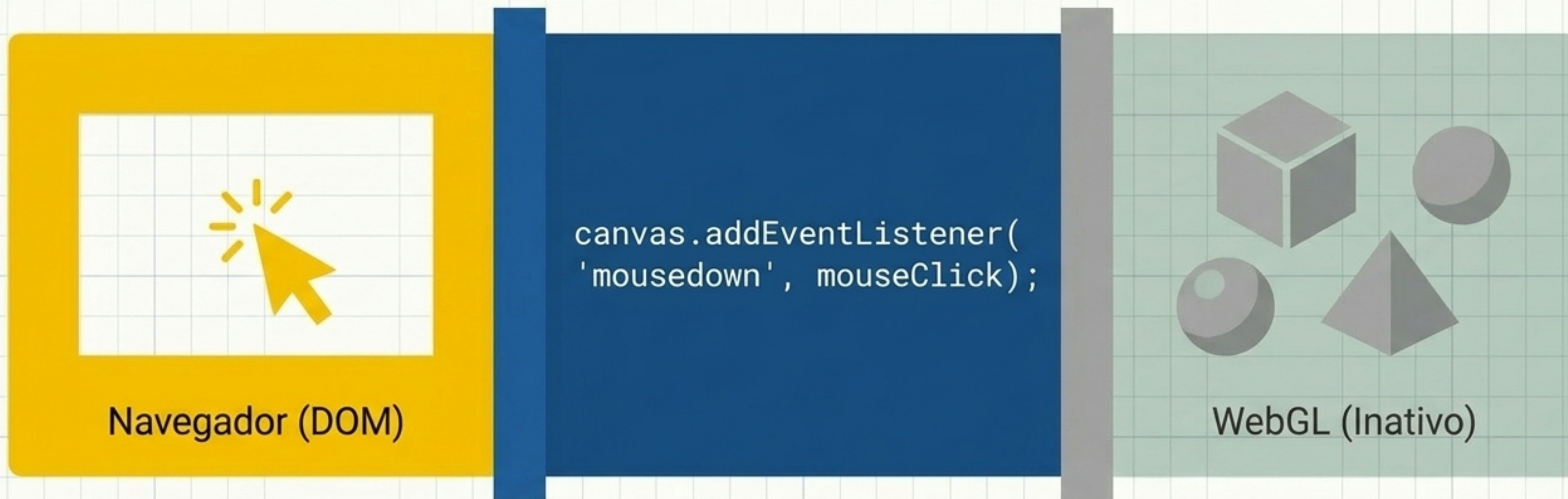


O Pipeline em Três Atos

Transformando uma ação física em uma instrução gráfica.

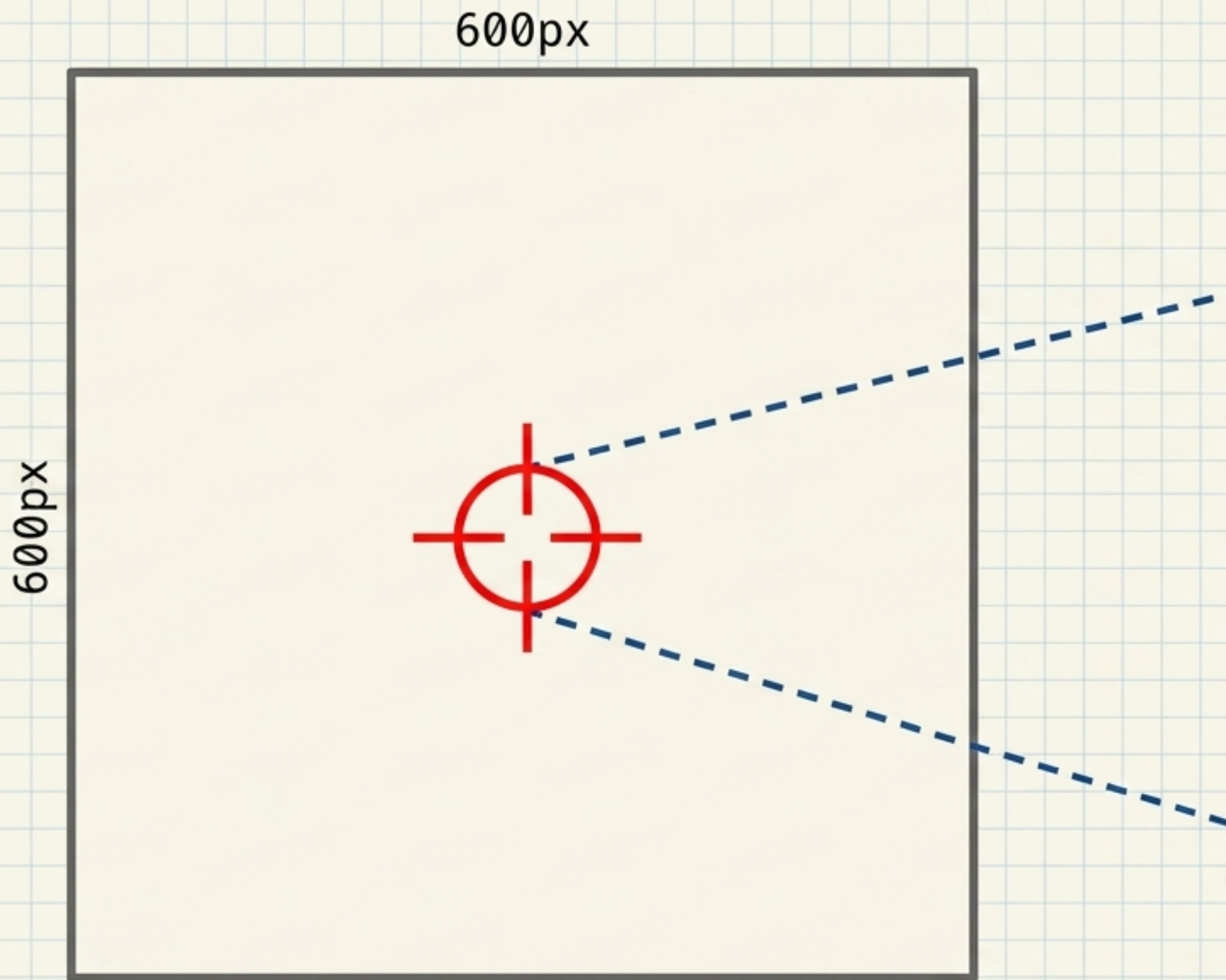


O Navegador Detecta o Clique



Você está dizendo ao navegador: "Quando acontecer um evento mousedown dentro deste canvas, execute a função". Esta é uma funcionalidade do DOM, não do WebGL. WebGL. O WebGL ainda não sabe que nada aconteceu.

O Objeto Event



Quando o **clique** ocorre, o navegador chama a função passando o objeto event.

evento.offsetX
evento.offsetY

Se clicarmos no centro de um canvas de 600px, teremos aproximadamente:

x = 300

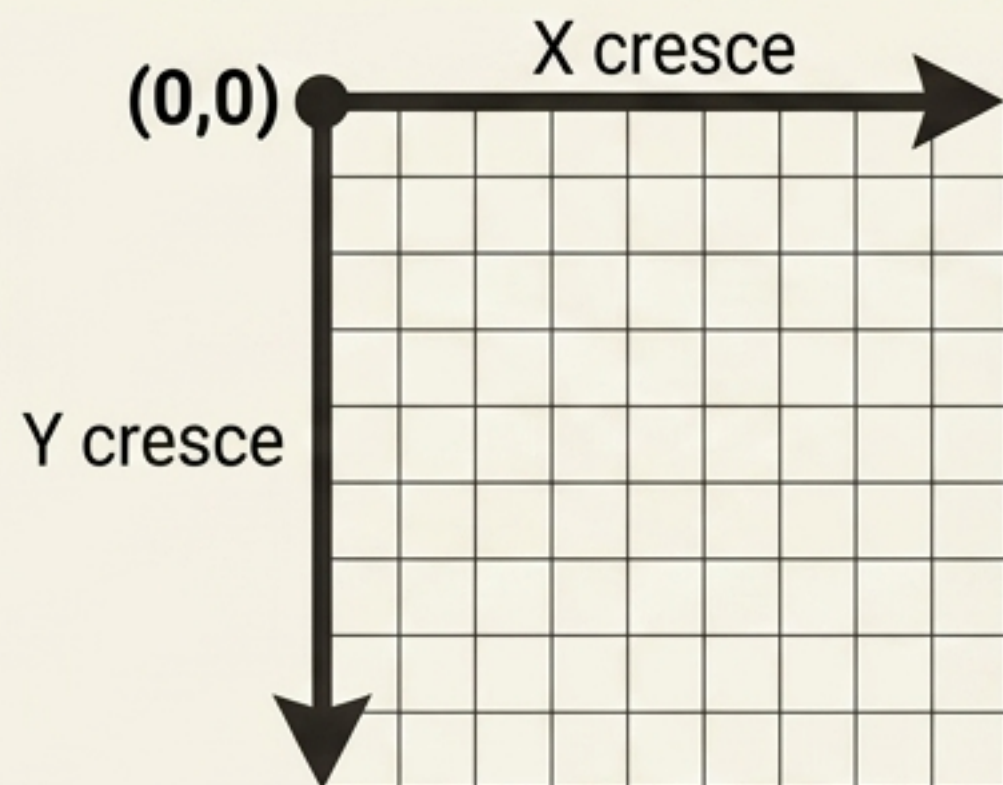
y = 300

Estas propriedades informam apenas onde ocorreu o clique no espaço físico da tela.

Dois Mundos, Duas Réguas

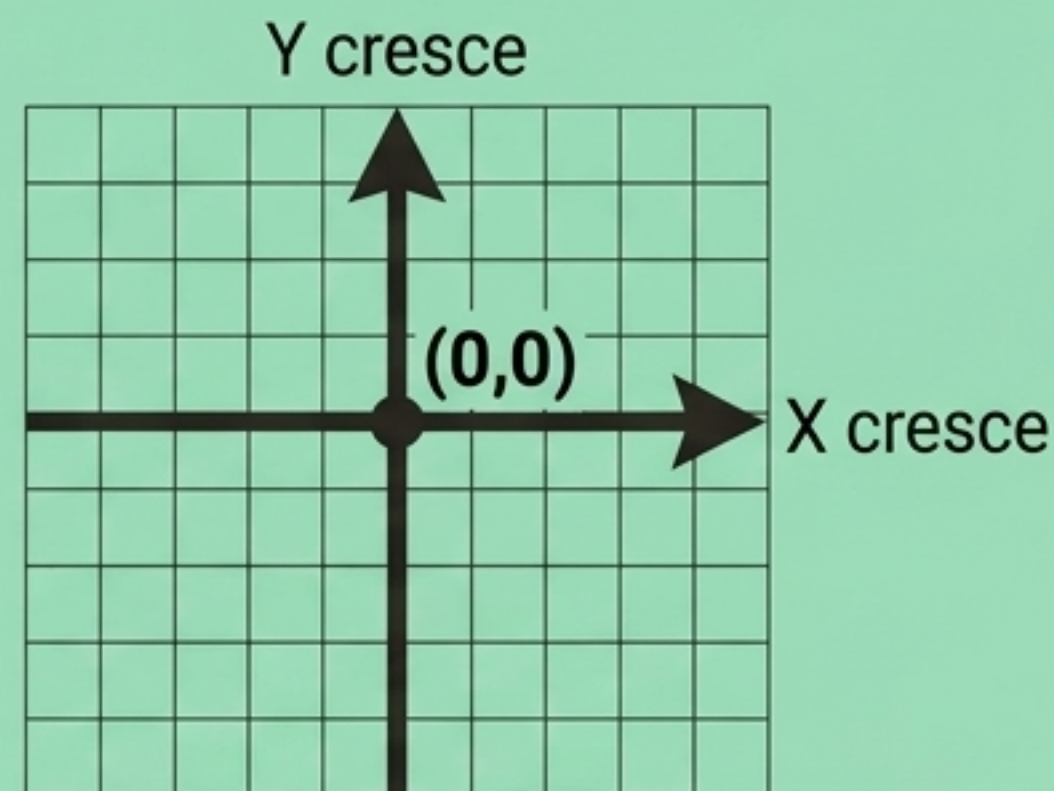
O Canvas e o WebGL utilizam sistemas de coordenadas fundamentalmente diferentes. Precisamos de uma conversão matemática para que a GPU entenda o clique.

Espaço Canvas (DOM)



- Unidade: Pixels (0 a 600)
- Origem: Canto superior esquerdo
- Eixo Y: Cresce para baixo

Espaço WebGL (GPU)



- Unidade: Coordenadas Normalizadas (-1 a +1)
- Origem: Centro absoluto
- Eixo Y: Cresce para cima

Normalizando o Eixo X

Convertendo de [0, 600] para [-1, +1].

```
let x = (evento.offsetX / 300) - 1;
```

Normaliza o valor para a metade da largura (0 a 2)

Desloca a origem para o centro (-1 a +1)

Exemplo no Centro:

$$x = (300 / 300) - 1$$
$$x = 1 - 1$$
$$x = 0 \text{ (Centro exato no WebGL)}$$

Invertendo o Eixo Y

Além de converter as proporções, o espaço WebGL exige que o eixo Y cresça para cima. O sinal negativo inverte o resultado final.

```
let y = -(evento.offsetY / 300) - 1;
```



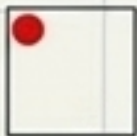
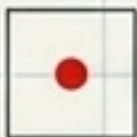
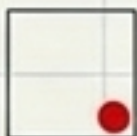
Inversão obrigatória do eixo

Exemplo no Topo do Canvas:

```
y = -((0 / 300) - 1)  
y = -(-1)  
y = 1 (Topo no WebGL)
```

A Matriz de Tradução

O resultado final da transformação matemática garante que qualquer pixel clicado encontre seu espaço exato na malha da GPU.

Posição na Tela	Canvas (DOM)	WebGL (GPU)
Clique no topo esquerdo 	$(0, 0)$	$(-1, +1)$
Clique no centro 	$(300, 300)$	$(0, 0)$
Clique embaixo à direita 	$(600, 600)$	$(+1, -1)$

Atualizando a Memória Local

Após a conversão, o código atualiza o array:

```
vertices[0] = x; vertices[1] = y;
```

Mas atenção: isso altera os dados **apenas na memória do JavaScript**. O WebGL ainda possui a **cópia antiga no seu buffer**. A tela não mudou.

Memória do JavaScript

[0.5, 0.5]



Buffer da GPU

[0.0, 0.0]

A Ponte para a GPU

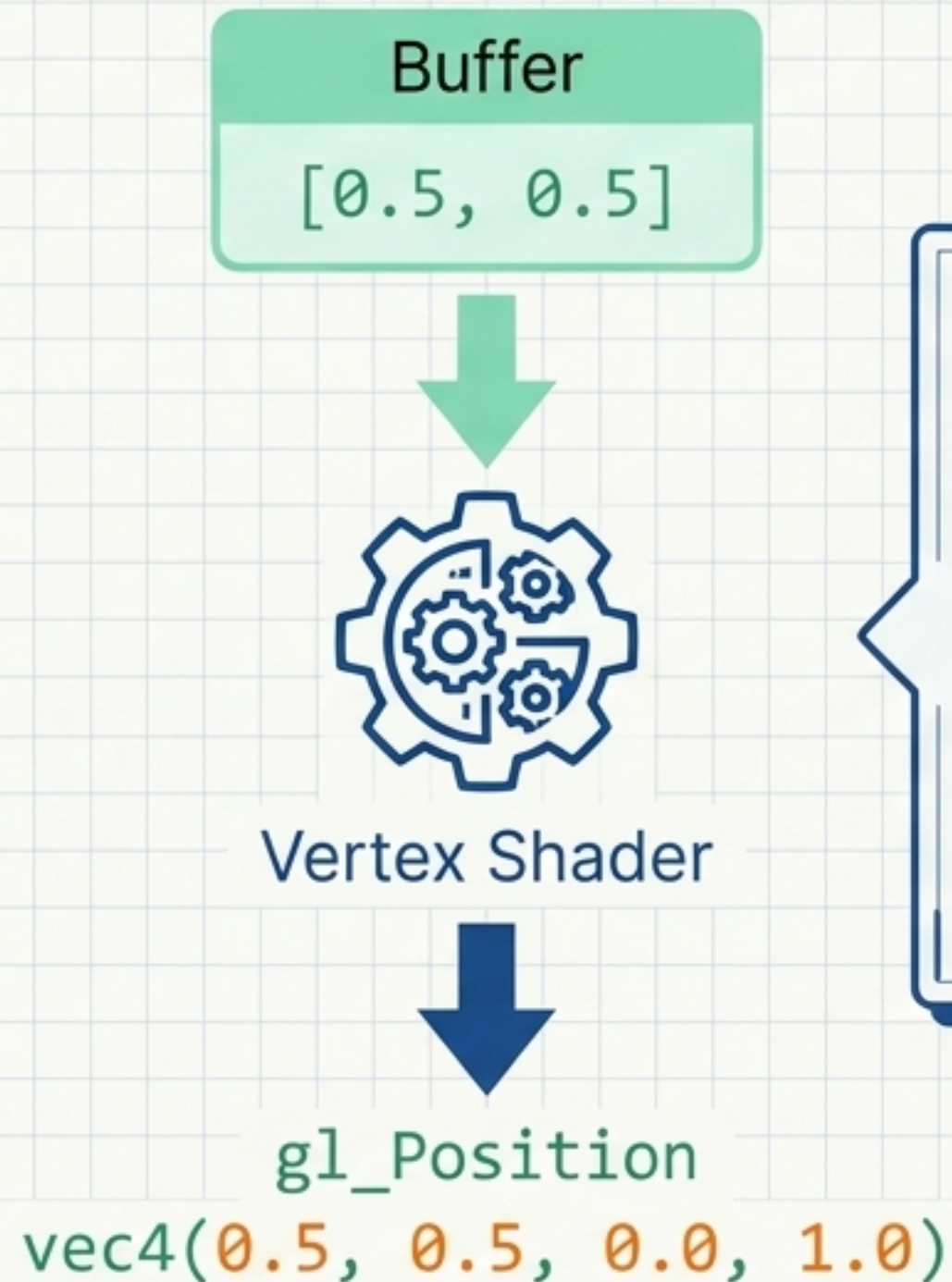
Para que a GPU enxergue a mudança, precisamos enviar os novos dados ativamente. O comando `bufferData` **empacota a nova posição e a transfere para o buffer utilizado pelo WebGL.**



```
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);  
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices), gl.STATIC_DRAW);
```

O Vértice Chega ao Shader

Dentro da GPU, o Vertex Shader recebe os dados do buffer atualizado e posiciona o vértice no espaço tridimensional.



`vertexPositionAttribute` vincula o buffer ao shader.

Quando o shader executa, ele lê a nova coordenada.

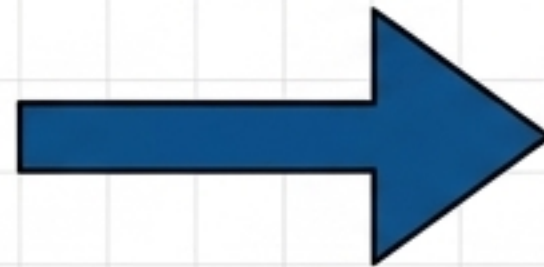
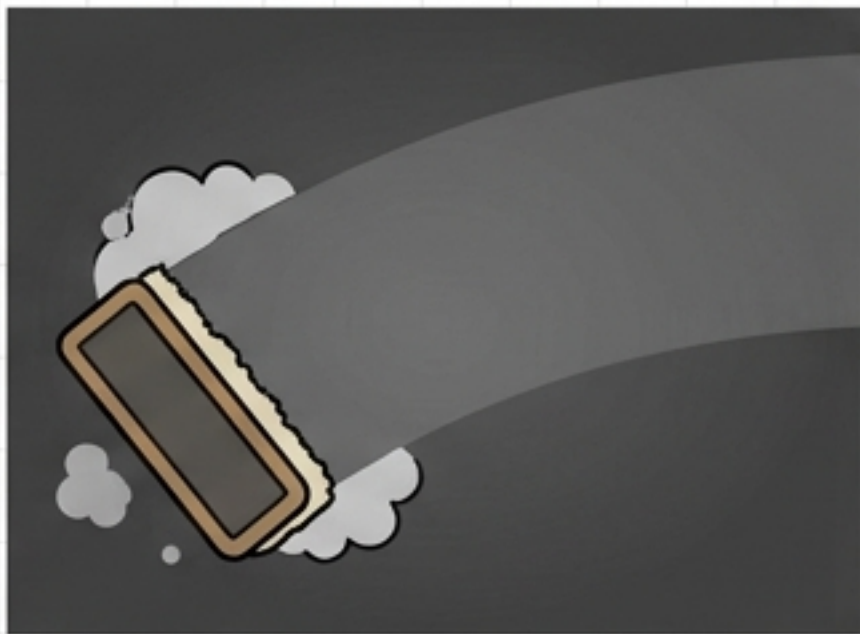
Se o clique produziu $x=0.5$, $y=0.5$, o shader fará:

```
gl_Position = vec4(0.5, 0.5, 0.0, 1.0);
```

O Redesenho

Com o buffer atualizado, comandamos a ação final. Primeiro, apagamos o desenho anterior. Depois, mandamos o WebGL desenhar os vértices novamente como pontos (**gl.POINTS**).

A Ordem de Desenho

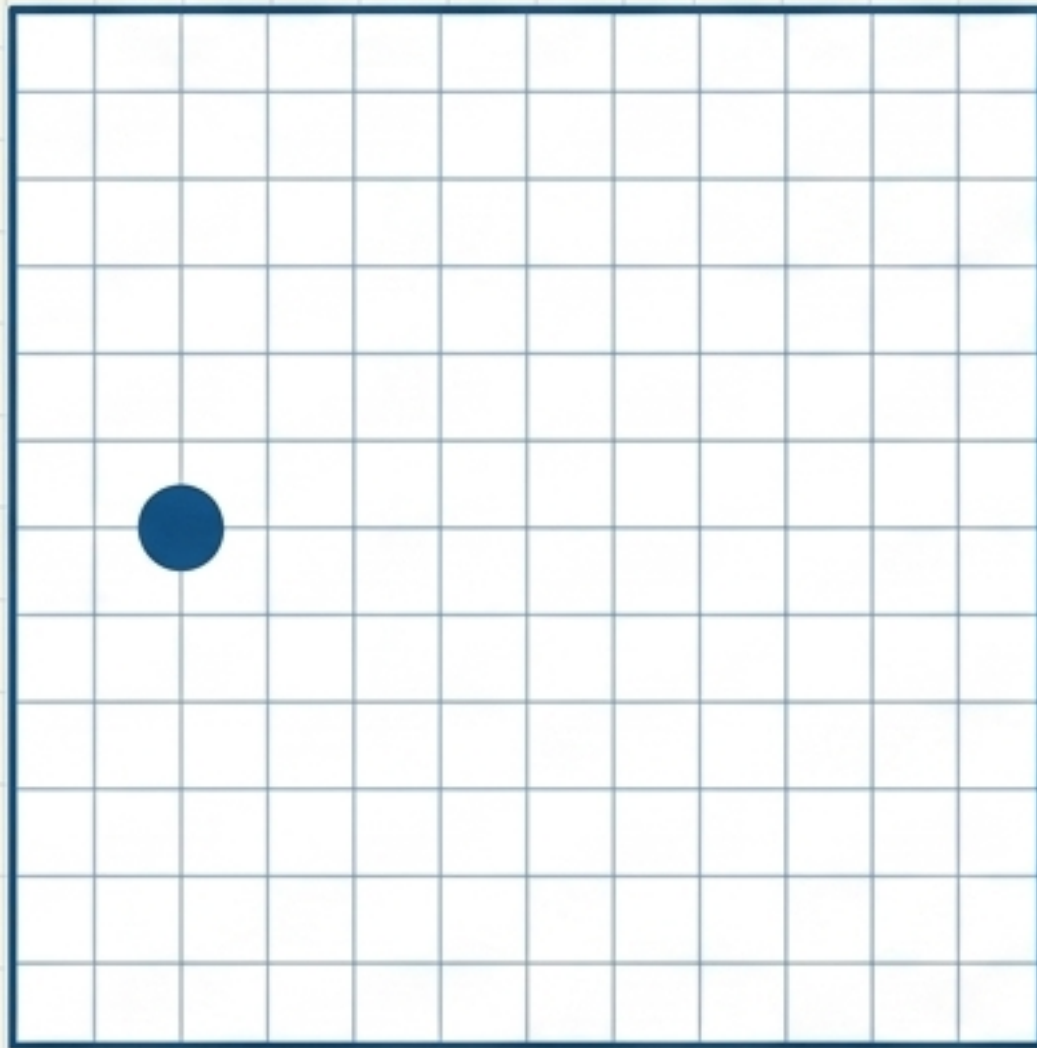


```
gl.clear(gl.COLOR_BUFFER_BIT);
```

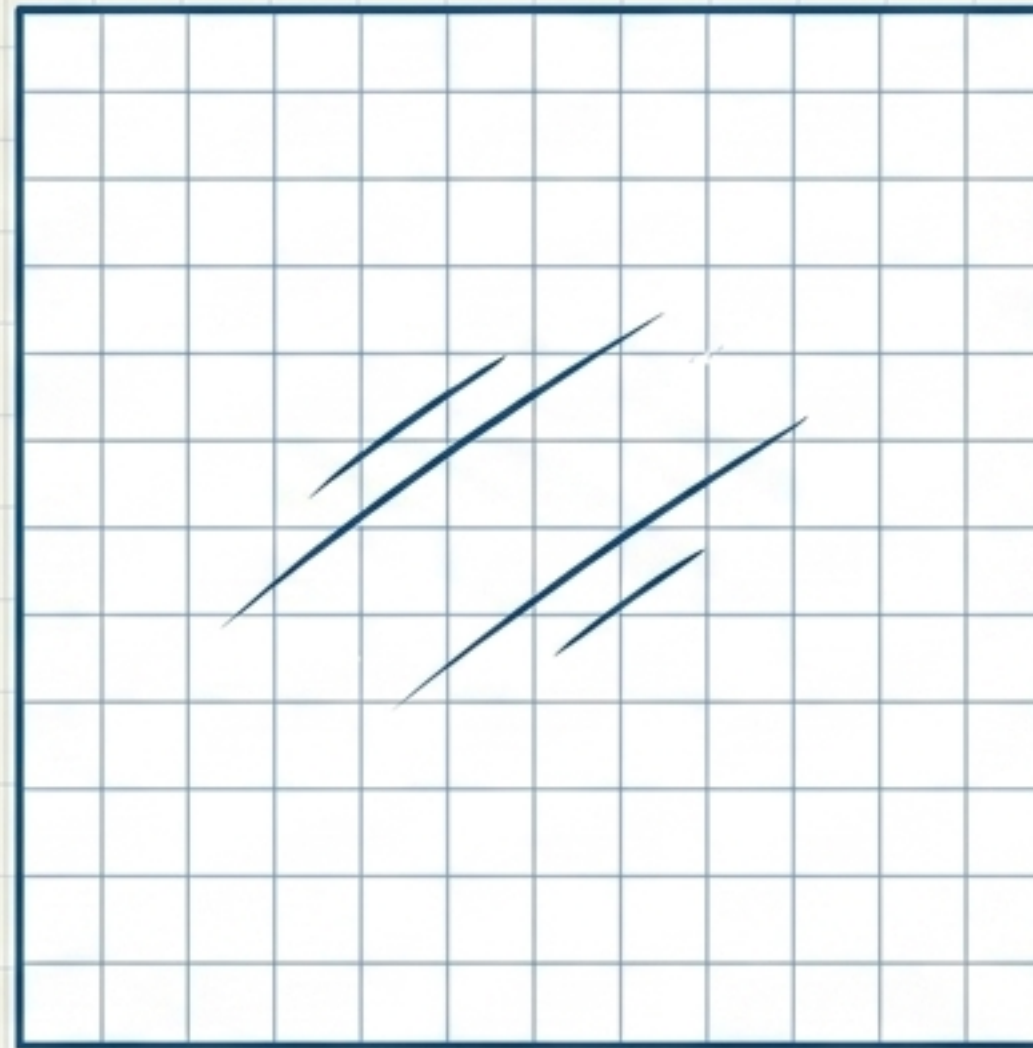
```
gl.drawArrays(gl.POINTS, 0, 1);
```

A Ilusão do Movimento

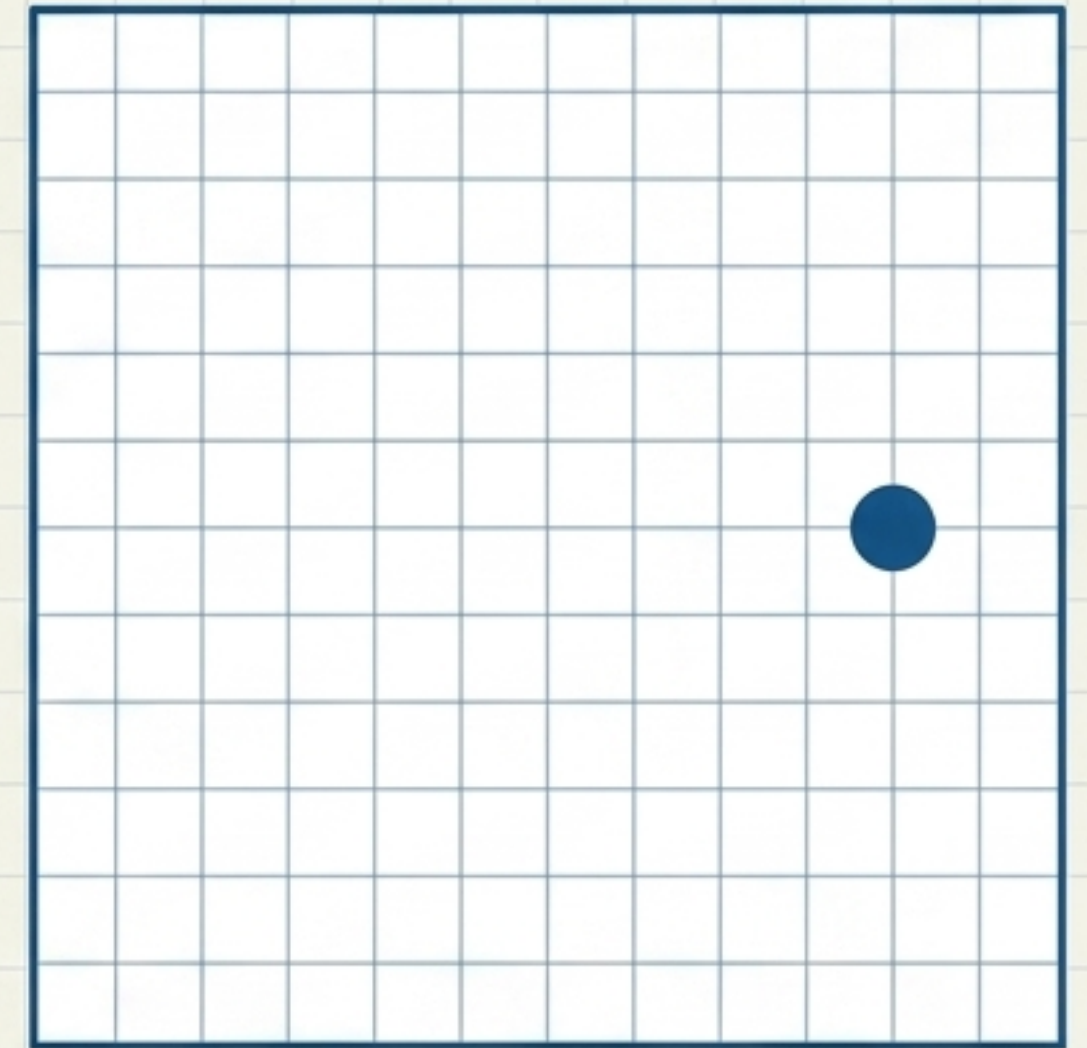
Uma regra de ouro da computação gráfica: não estamos pegando um ponto e o “movendo” pela tela. O ponto original não foi deslocado. Ele foi sumariamente apagado, e um ponto inteiramente novo foi desenhado na nova coordenada.



Estado Anterior



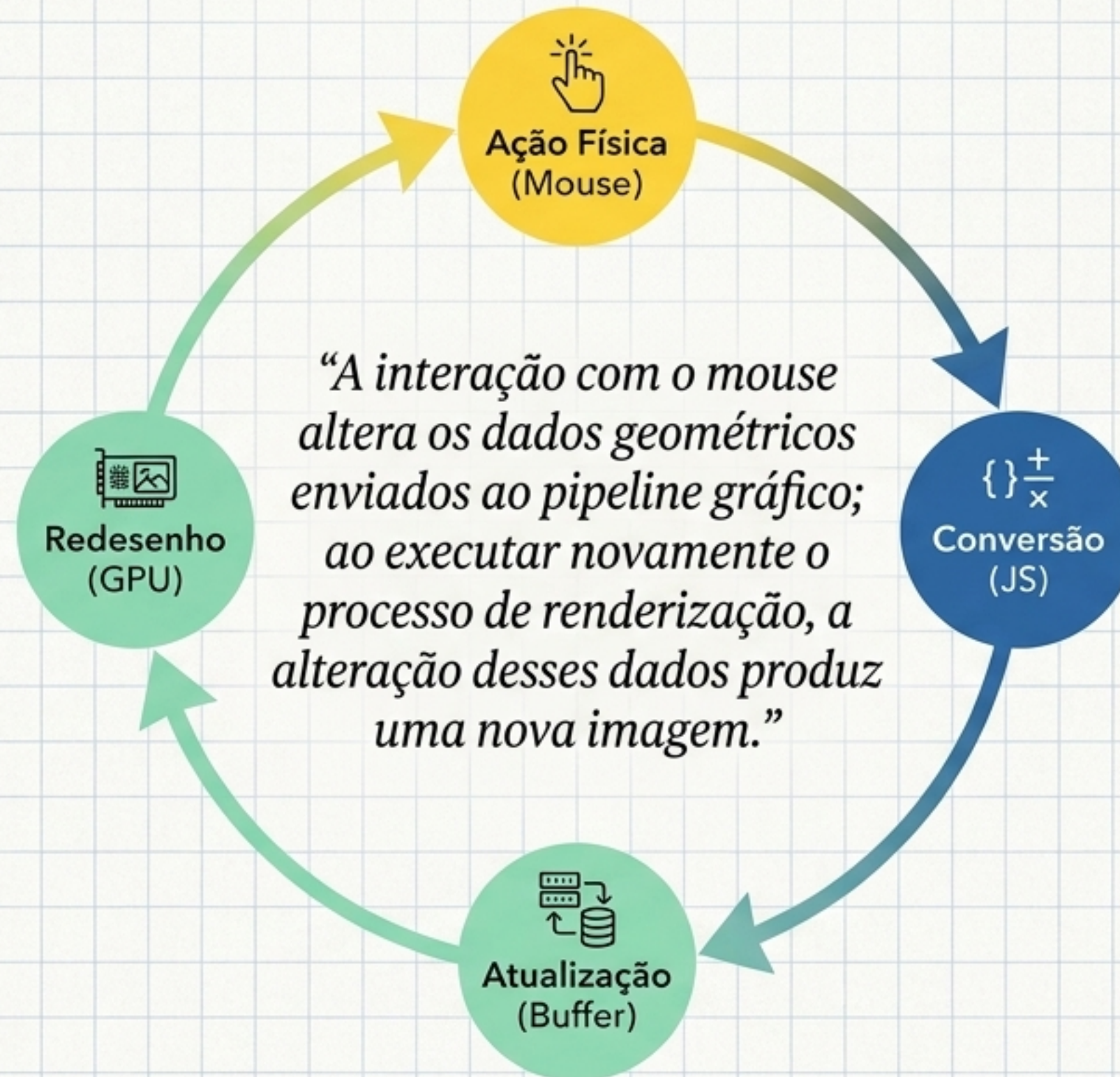
O Apagamento (Wipe)



O Novo Desenho

O Ciclo Completo

A essência da interatividade no WebGL.



O Próximo Desafio

Agora que sabemos como renderizar um ponto a partir de um clique, o que acontece se capturarmos dois cliques?
Na próxima etapa: Usando dois vértices no espaço WebGL para aplicar o Algoritmo de Bresenham e desenhar uma linha, pixel por pixel.

